

🔥 SUPER PROMPT — AI SERVICE ORCHESTRATOR FOR INFORMAL ECONOMY

Hackathon-Grade, Bug-Free, Full-Stack Execution Blueprint

⚡ MASTER DIRECTIVE (READ THIS FIRST)

You are building a **production-quality, agentic AI system** for the informal service economy of Pakistan. This is NOT a CRUD app. This is NOT a listing app. This is a **multi-agent reasoning system** that thinks, decides, books, and follows up — autonomously.

You must build this with **Google Antigravity at its core**. Every agent action, every tool call, every reasoning step **MUST** flow through Antigravity's orchestration layer.

Before writing a single line of code, you MUST:

1. Confirm the full tech stack is ready
 2. Confirm Google Antigravity SDK/API is set up and authenticated
 3. Confirm all environment variables are defined in `.env`
 4. Confirm mock data files are seeded and readable
 5. Confirm the folder structure matches the architecture below
 6. Only then proceed — and after each module is built, **re-run it and fix every error before moving on**
-

🏗️ SYSTEM ARCHITECTURE (NON-NEGOTIABLE)

```
/project-root
├── /agents
│   ├── intent_agent.py           # NLP: parses Urdu/Roman Urdu/English
│   ├── discovery_agent.py        # Finds providers via Maps/mock DB
│   ├── ranking_agent.py          # Scores and ranks providers
│   ├── booking_agent.py          # Simulates booking + confirmation
│   ├── followup_agent.py         # Schedules reminders + status updates
│   └── orchestrator.py           # Antigravity-powered master controller
├── /tools
│   ├── maps_tool.py              # Google Maps / Places API wrapper
│   ├── search_tool.py            # Google Search API tool
│   └── nlp_tool.py               # Language detection + entity extraction
```

```
|   ├── db_tool.py           # Mock DB read/write operations
|   └── notification_tool.py  # Simulated SMS/WhatsApp notifications
|
|── /data
|   ├── providers.json       # Mock provider dataset (50+ entries)
|   ├── bookings.json        # Live booking state (read/write)
|   ├── service_categories.json # Taxonomy of services
|   └── locations_pk.json     # Pakistan locality data (sectors, areas)
|
|── /mobile_app               # React Native or Flutter (MANDATORY)
|   ├── screens/
|   ├── components/
|   └── services/api.js
|
|── /web_app (optional)       # React + Tailwind
|
|── /logs
|   └── agent_trace.jsonl     # Append-only agent reasoning logs
|
|── /tests
|   ├── test_intent.py
|   ├── test_booking.py
|   └── test_e2e.py
|
|── .env                      # ALL secrets here, never hardcoded
|── README.md
└── requirements.txt / package.json
```

AGENT DEFINITIONS — BUILD EACH EXACTLY AS DESCRIBED

AGENT 1: Intent Agent

Purpose: Understand what the user wants from any language input.

SYSTEM PROMPT FOR THIS AGENT:

"You are an expert at understanding service requests in Pakistani context.

Extract from the input:

- service_type: (AC repair, plumber, tutor, electrician, beautician, etc.)
- location: (sector, area, city – normalize to standard format)
- time_preference: (today, tomorrow morning, specific time, ASAP, etc.)
- urgency_level: (emergency, normal, flexible)
- language_detected: (Urdu, Roman Urdu, English, Mixed)

Always return valid JSON. If a field is missing, use null.

Handle Roman Urdu like: 'kal subah', 'abhi chahiye', 'G-13 mein'

Handle Urdu script natively.

Never hallucinate a field – only extract what is present."

Inputs: Raw user text string **Outputs:** Structured JSON `{service_type, location, time_preference, urgency_level, language_detected}` **Error handling:** If extraction confidence < 0.7, return clarification_needed: true with specific question **Test cases to pass:**

- "Mujhe kal subah G-13 mein AC technician chahiye" ✓
- "plumber needed asap in F-10" ✓
- "مجھے آج شام کو الیکٹریشن چاہیے" ✓
- "I need a tutor for my kid in I-8 sector tomorrow" ✓
- empty string → ask for input ✓
- gibberish → ask for clarification ✓

AGENT 2: Discovery Agent

Purpose: Find relevant service providers near the user's location.

SYSTEM PROMPT FOR THIS AGENT:

"You are a provider discovery specialist for Pakistan's informal economy.

Given a service_type and location, find the best matching providers.

Search the provider database first.

If the database has fewer than 3 results, augment with Google Places API.

Always return providers within 10km radius unless none exist.

Return list sorted by distance (raw, unranked – ranking is handled by Agent 3).

Include: provider_id, name, phone, location_coords, distance_km,
service_categories[], rating, availability_slots[], verified"

Priority lookup order:

1. Local mock database ((/data/providers.json))
2. Google Places API (real call or mocked)
3. Fallback: nearest providers regardless of exact service match

Must handle: zero results → suggest adjacent categories

AGENT 3: Ranking Agent

Purpose: Intelligently score and rank providers.

Scoring Formula (MUST implement this exactly):

python

```
def calculate_score(provider, request):
    distance_score = max(0, 10 - provider['distance_km']) * 0.35 # 35% weight
    rating_score = (provider['rating'] / 5.0) * 10 * 0.30 # 30% weight
    availability_score = (1 if request['time'] in provider['slots']) * 10 * 0.25 #
    verified_bonus = 1.5 if provider['verified'] else 0 # bonus
    total = distance_score + rating_score + availability_score + verified_bonus
    return round(total, 2)
```

Output: Ranked list with reasoning string per provider

json

```
{
  "rank": 1,
  "provider": "Ali AC Services",
  "score": 8.7,
  "reasoning": "Closest verified provider (2.1km), 4.8★ rating, available at request"
}
```

AGENT 4: Booking Agent

Purpose: Simulate the complete booking lifecycle.

CRITICAL — Must perform ALL of these steps:

```
Step 1: SELECT top-ranked provider
Step 2: CHECK slot availability (read from bookings.json)
Step 3: RESERVE slot (write to bookings.json atomically)
Step 4: GENERATE booking_id (format: BK-YYYYMMDD-XXXX)
Step 5: CREATE confirmation message in user's language
Step 6: LOG booking to agent_trace.jsonl
Step 7: TRIGGER notification simulation (SMS/WhatsApp mock)
Step 8: RETURN booking receipt object
```

Booking Receipt Schema:

```
json
{
  "booking_id": "BK-20250518-4821",
  "status": "CONFIRMED",
  "service": "AC Repair",
  "provider": {
    "name": "Ali AC Services",
    "phone": "+92-300-1234567",
    "distance": "2.1 km"
  },
  "slot": {
    "date": "2025-05-19",
    "time": "10:00 AM",
    "duration_estimate": "1-2 hours"
  },
  "location": "G-13, Islamabad",
  "confirmation_message": "Aapki booking confirm ho gayi! Ali AC Services kal 10 baj",
  "follow_up_scheduled": true,
  "created_at": "2025-05-18T14:32:00Z"
}
```

AGENT 5: Follow-Up Agent

Purpose: Automate post-booking lifecycle.

Must simulate:

- T-1hr: Reminder notification to user
- T-30min: Provider "on the way" status update

- T+0: Service start confirmation
- T+service_duration: Completion prompt + rating request

All follow-ups write to:

- `bookings.json` (status field update)
 - `agent_trace.jsonl` (full log)
 - Console/UI notification simulation
-

ORCHESTRATOR (The Heart — Antigravity-Powered)

```
python
```

```
# orchestrator.py – ANTIGRAVITY CORE
```

```
"""
```

```
ANTIGRAVITY ORCHESTRATION INSTRUCTIONS:
```

```
This orchestrator must:
```

1. Use Antigravity's multi-step reasoning pipeline
2. Pass context between agents using Antigravity's shared memory/state
3. Use Antigravity's tool registry to call Maps, Search, DB tools
4. Maintain full planning trace visible in Antigravity's dashboard
5. Handle failures with Antigravity's retry + fallback logic

```
Pipeline flow (MUST be implemented in this exact order):
```

```
[User Input]
```

- Antigravity receives intent
- PLAN: identify required agents
- EXECUTE: Intent Agent (extract structured data)
- EXECUTE: Discovery Agent (find providers)
- EXECUTE: Ranking Agent (score + select)
- DECIDE: confirm with user OR auto-book (based on urgency)
- EXECUTE: Booking Agent (simulate booking)
- EXECUTE: Follow-Up Agent (schedule reminders)
- OUTPUT: Full receipt + reasoning trace
- LOG: Write complete trace to agent_trace.jsonl

```
At each step, Antigravity must:
```

- Log the agent name + input + output
- Record tool calls made
- Track decision rationale
- Handle errors with graceful fallback

```
"""
```

MOCK DATA REQUIREMENTS

`/data/providers.json` — Must have **minimum 30 providers** covering:

```
json
```

```
[
  {
    "id": "PRV-001",
    "name": "Ali AC Services",
    "owner": "Ali Hassan",
    "phone": "+92-300-1234567",
    "whatsapp": "+92-300-1234567",
    "service_categories": ["AC repair", "AC installation", "AC cleaning"],
    "location": {
      "area": "G-13",
      "city": "Islamabad",
      "lat": 33.6844,
      "lng": 73.0479
    },
    "rating": 4.8,
    "total_reviews": 127,
    "verified": true,
    "experience_years": 8,
    "availability": {
      "days": ["Mon", "Tue", "Wed", "Thu", "Fri", "Sat"],
      "slots": ["09:00", "10:00", "11:00", "14:00", "15:00", "16:00"]
    },
    "price_range": "PKR 500-2000",
    "response_time_avg": "15 mins",
    "languages": ["Urdu", "Punjabi"]
  }
  // ... 29 more with varied services, locations, ratings
]
```

Service categories to cover (minimum): AC Technician, Plumber, Electrician, Carpenter, Painter, Tutor (Math/English/Science), Beautician, Mehendi Artist, Home Cleaner, Cook/Chef, Driver, Tailor, Welder, Computer Repair, Mobile Repair, Generator Repair, Water Tank Cleaning

MOBILE APP REQUIREMENTS (MANDATORY)

Framework: React Native (Expo) OR Flutter

Screens to build:

1. Home/Chat Screen

- WhatsApp-style chat interface
- Supports text input in any language

- Voice input button (UI only if not implemented)
- Shows typing indicator while agents process

2. Provider Cards Screen

- Card per provider with: name, rating stars, distance, price range
- "Book Now" and "View Details" CTAs
- Color-coded availability badge

3. Booking Confirmation Screen

- Full receipt display
- Countdown timer to appointment
- Map showing provider location (Google Maps embed)
- Share/Save booking option

4. Booking History Screen

- List of past + upcoming bookings
- Status badges: CONFIRMED, IN PROGRESS, COMPLETED, CANCELLED
- Tap to expand full details

5. Agent Trace Viewer (DEV MODE)

- Collapsible view of reasoning steps
- Shows each agent's input/output
- Tool call logs
- Timestamps

UI/UX Rules:

- Primary color:  #00A884 (WhatsApp green — familiar to target users)
- Support RTL text rendering for Urdu
- Large touch targets (min 48px)
- Offline-capable with cached provider data
- Loading states for every async operation
- Error states with retry buttons

GOOGLE ANTIGRAVITY INTEGRATION — SECRET TIPS

These are the implementation patterns that maximize your Antigravity score (25% of evaluation):

python

```
# Don't just call agents sequentially – make Antigravity PLAN first
antigravity.plan(
    goal="Book a service for the user",
    available_agents=["intent", "discovery", "ranking", "booking", "followup"],
    context={"user_input": raw_text, "timestamp": now}
)
# This generates a visible plan trace – evaluators LOVE this
```

Tip 2: Tool Registration (Maps + Search + DB as Antigravity Tools)

python

```
# Register ALL external calls as Antigravity tools, not raw API calls
@antigravity.tool(name="google_maps_search", description="Find nearby service providers")
def maps_search(query: str, location: dict, radius_km: int) -> list:
    # Your Google Maps API call here
    pass

# This makes tool usage traceable in Antigravity dashboard
```

Tip 3: Shared State Between Agents

python

```
# Use Antigravity's shared context object – NOT global variables
state = antigravity.create_context()
state.set("user_request", parsed_intent)
state.set("candidates", discovered_providers)
# Each agent reads/writes to this – creates clean audit trail
```

Tip 4: Force Reasoning Visibility

python

```
# Each agent MUST emit reasoning before acting
agent.emit_reasoning(
    step="Provider Ranking",
    thought="Ali AC Services scores highest because: 2.1km distance (closest), 4.8★
    confidence=0.92
)
```

Tip 5: Antigravity Retry + Fallback Pattern

```
python

@antigravity.with_retry(max_attempts=3, fallback=use_mock_data)
def call_google_places(service, location):
    # If API fails → Antigravity auto-uses mock data
    # This demonstrates robustness to evaluators
    pass
```

Tip 6: Multi-Agent Demonstration (Evaluators Watch For This)

```
python

# Explicitly show agents "talking" through Antigravity message bus
antigravity.send_message(
    from_agent="discovery_agent",
    to_agent="ranking_agent",
    payload={"providers": candidates, "request": user_request}
)
# This proves multi-agent architecture, not just function calls
```

✅ SELF-VERIFICATION CHECKLIST (RUN AFTER EVERY MODULE)

After building each component, run these checks before moving on:

INTENT AGENT CHECKS:

- Handles English input correctly
- Handles Roman Urdu correctly (test: "kal subah AC wala chahiye F-8 mein")
- Handles Urdu script (test: "مجھے آج الیکٹریشن چاہیے")
- Returns valid JSON always
- Handles empty/gibberish input gracefully
- Logs to agent_trace.jsonl

DISCOVERY AGENT CHECKS:

- Reads from providers.json correctly
- Filters by service category
- Filters by location (within radius)
- Returns ≥ 1 result for all major service types
- Falls back to Google Places API if < 3 results
- Handles zero results without crashing

RANKING AGENT CHECKS:

- Scoring formula implemented correctly
- Distance weight = 35%, Rating = 30%, Availability = 25%, Bonus = 10%
- Returns ranked list with scores
- Provides reasoning string per provider
- Deterministic – same input = same ranking

BOOKING AGENT CHECKS:

- Generates unique booking ID
- Writes to bookings.json atomically (no corruption)
- Creates confirmation message in user's language
- Logs to agent_trace.jsonl
- Returns complete booking receipt object
- Handles "slot already taken" scenario

FOLLOWUP AGENT CHECKS:

- Schedules T-1hr reminder
- Schedules T-30min status update
- Updates booking status in bookings.json
- All events logged to agent_trace.jsonl

ORCHESTRATOR CHECKS:

- Antigravity plan is generated before execution
- All agents called in correct order
- Context passed correctly between agents
- Tool calls registered in Antigravity
- Complete trace visible in Antigravity dashboard

- End-to-end test passes for all 4 demo scenarios

MOBILE APP CHECKS:

- App launches without errors
- Chat screen sends request and receives response
- Provider cards display correctly
- Booking confirmation screen shows complete receipt
- Map shows provider location
- RTL text works for Urdu
- All loading/error states implemented
- Works on both iOS and Android simulators

FINAL SYSTEM CHECKS:

- Complete e2e flow works: "Mujhe kal subah 6-13 mein AC technician chahiye"
- agent_trace.jsonl has full reasoning log
- No hardcoded API keys (all in .env)
- README documents all architecture decisions
- Demo video script covers all mandatory requirements

🚫 ABSOLUTE DON'TS

- **NEVER** build a simple listing app — every step must show autonomous decision-making
- **NEVER** hardcode provider selection — ranking must be computed
- **NEVER** skip the agent_trace.jsonl logging — evaluators look for this
- **NEVER** call Google APIs directly — always wrap as Antigravity tools
- **NEVER** use global variables for agent state — use Antigravity's context object
- **NEVER** skip error handling — every agent must have fallback behavior
- **NEVER** store API keys in code — use .env always
- **NEVER** skip the follow-up simulation — it's a mandatory requirement
- **NEVER** present only 1 provider option — always show top 3 with ranked reasoning
- **NEVER** use fake/hardcoded booking IDs — generate them dynamically

🏆 WINNING EXTRAS (Bonus Points)

These will push your project from good to exceptional:

1. **Voice Input** — Use Google Speech-to-Text via Antigravity to accept spoken Urdu requests
2. **WhatsApp Simulation** — Show a mock WhatsApp conversation in the UI (not real API)

3. **Live Map Animation** — Show provider "moving" toward user location post-booking
 4. **Multi-booking** — Handle "I need both a plumber AND electrician"
 5. **Conflict Resolution** — If top provider is busy, auto-suggest next best with explanation
 6. **Provider Rating Flow** — After service completion, collect a star rating
 7. **Booking Analytics Dashboard** — Show stats: bookings today, avg rating, popular services
 8. **Multilingual Receipts** — Receipt generated in same language as user's request
 9. **Smart Clarification** — If location is ambiguous ("F sector"), ask "F-6, F-7, F-8, or F-10?"
 10. **Antigravity Dashboard Link** — Show live Antigravity trace URL in the demo
-

DEMO VIDEO SCRIPT (3-5 minutes)

Minute 1: Problem + Architecture (30s problem, 30s architecture diagram)

- Show the challenge statement
- Display your system architecture diagram
- Highlight Antigravity as the central orchestration layer

Minute 2: Live Demo — Intent to Discovery

- Type: "Mujhe kal subah G-13 mein AC technician chahiye"
- Show agent_trace in real time as intent is extracted
- Show provider discovery happening (maps search animation)

Minute 3: Ranking + Booking Simulation

- Show 3 ranked providers with scores and reasoning
- Select top provider → trigger booking simulation
- Show booking receipt being generated
- Show bookings.json being updated in real time

Minute 4: Follow-Up + Multi-language

- Show follow-up scheduler activating
- Show T-1hr reminder notification mock
- Repeat demo in English to show multilingual capability

Minute 5: Agent Trace + Antigravity Dashboard

- Show agent_trace.jsonl with full reasoning
- Open Antigravity dashboard showing planning + execution trace

- Show multi-agent message passing
 - End with booking history screen
-

README TEMPLATE

markdown

AI Service Orchestrator – Informal Economy

System Architecture

[Diagram image]

Google Antigravity Integration

- Used for: multi-agent orchestration, tool registration, shared state, planning
- Agents registered: Intent, Discovery, Ranking, Booking, FollowUp
- Tools registered: google_maps, google_search, mock_db, notification_sim

Agent Pipeline

User Input → Intent Agent → Discovery Agent → Ranking Agent → Booking Agent → FollowUp Agent

APIs Used

- Google Antigravity (core orchestration)
- Google Maps / Places API (provider discovery)
- Google Search API (supplementary discovery)
- Google Speech-to-Text (optional voice input)

Tech Stack

- Backend: Python 3.11, FastAPI
- Mobile: React Native (Expo) / Flutter
- Database: JSON files (mock) / Firebase (optional)
- Orchestration: Google Antigravity

Setup

1. Clone repo
2. Copy .env.example to .env and fill keys
3. pip install -r requirements.txt
4. python orchestrator.py
5. cd mobile_app && expo start

Assumptions

- Provider data is mocked; real implementation would use verified database
- Booking simulation does not connect to real payment gateway
- Notifications are simulated (no real SMS/WhatsApp API used)

Limitations

- Real-time availability requires provider-side app
- Voice input accuracy depends on network quality

ENVIRONMENT VARIABLES (.env)

```
bash

# Google APIs
GOOGLE_ANTIGRAVITY_API_KEY=your_key_here
GOOGLE_MAPS_API_KEY=your_key_here
GOOGLE_PLACES_API_KEY=your_key_here
GOOGLE_SPEECH_API_KEY=your_key_here

# App Config
APP_ENV=development
LOG_LEVEL=debug
BOOKING_ID_PREFIX=BK
DEFAULT_SEARCH_RADIUS_KM=10
DEFAULT_CITY=Islamabad
DEFAULT_COUNTRY=PK

# Mock Flags (set to true if APIs unavailable)
USE MOCK_MAPS=true
USE MOCK_PROVIDERS=true
USE MOCK_NOTIFICATIONS=true
```

END OF SUPER PROMPT

Build in order. Test after each step. Never skip the checklist. Log everything. The evaluators are looking for: Antigravity traces, multi-agent reasoning, and booking simulation. Win by showing a system that THINKS, not just one that displays data.